STUDENT PLACEMENT MANAGEMENT SYSTEM

Project Documentation

-[Deepak L]

1. Project Overview

The Student Placement Management System is a backend REST API application developed using Java and Spring Boot. It provides a secure platform for managing students, companies, job drives, applications, interviews and offers. MySQL is used for persistent data storage, while Spring Security (HTTP Basic with BCrypt) authenticates users and controls access based on their roles.

2. Objectives

- Maintain student and company records.
- Allow companies and admins to create job drives with eligibility criteria.
- Allow students to apply for job drives with automatic eligibility checking.
- Track each application from APPLIED to SELECTED or REJECTED.
- Schedule interviews and record results and feedback.
- Create offers and track their status.
- Close job drives automatically after the application deadline.
- Provide validation, centralized exception handling and Swagger/OpenAPI documentation.

3. User Roles

Role	Main responsibilities
ADMIN	Full access: manage students, companies, drives, applications, interviews and offers.
STUDENT	Manage student profile, view job drives, apply, view interviews and offers.
COMPANY	Manage company profile and job drives, review applications, schedule interviews, issue offers.

4. Technologies

Technology	Use
Java 17	Application development
Spring Boot 4.0.0	Backend REST application
Spring Data JPA / Hibernate	Database ORM
MySQL	Database (`placement_db`)
Spring Security	Authentication and authorization
BCrypt	Password hashing
Spring Cache	Student data caching
Spring Scheduler	Background jobs
Maven	Build and dependency management
Swagger / Springdoc	API documentation and testing
JUnit, JaCoCo	Unit testing and coverage

5. System Architecture

The project follows a layered architecture. Each layer has a specific responsibility, making the application easier to maintain, test and extend.

```
Client (Postman / Swagger / Frontend)
        ↓
Controller Layer
        ↓
Service Layer
        ↓
Repository Layer
        ↓
MySQL Database
```

6. Project Structure

Base package: `com.springboot.placement`

Package	Purpose
config	Security, OpenAPI, auditing and scheduling configuration.
controller	REST API endpoints.
service	Business logic (interfaces and implementations).
repository	Database operations using Spring Data JPA.
entity	JPA entities mapped to database tables.
dto	Request and response objects with validation.
mapper	Entity ↔ DTO conversion.
exception	Custom exceptions and global exception handling.
enums	Role and status values.
scheduler	Scheduled background jobs.
security	Loads users from the database for login.
constants	Common error messages.

7. Main Entities

Entity	Purpose
Student	Stores name, email, phone, college, degree, branch, CGPA, graduation year and role.
Company	Stores company name, email, phone, location and industry.
JobDrive	Stores title, description, minimum CGPA, required branch, location, drive date, deadline and status.
Application	Stores a student's application to a job drive, with date and status.
Interview	Stores interview date, mode, result and feedback.
Offer	Stores offer date, salary, joining date and status.
User	Stores login username, hashed password and role.

All main entities extend an `Audit` class that records created and updated date and user, plus a version field.

8. Important Relationships

- One Company can have many JobDrives.
- Students and JobDrives have a many-to-many relationship (`student_job_drive`).
- One Student can have many Applications.
- One JobDrive can have many Applications.
- One Application has one Interview.
- One Application has one Offer.

Status values:

Enum	Values
JobDriveStatus	OPEN, CLOSED
ApplicationStatus	APPLIED → SHORTLISTED → INTERVIEW → SELECTED / REJECTED
InterviewMode / InterviewResult	ONLINE, OFFLINE / PENDING, PASSED, FAILED
OfferStatus	PENDING, ACCEPTED, REJECTED

9. Authentication and Security

Authentication is implemented using Spring Security with HTTP Basic and BCrypt. Passwords are stored as BCrypt hashes, never as plain text.

```
Register user (role assigned)
        ↓
Client sends username and password with each request
        ↓
CustomUserDetailsService loads user from database
        ↓
BCrypt verifies password
        ↓
Role authorization
        ↓
Controller
```

Unauthenticated requests receive a JSON response with status 401, and requests without permission receive 403.

10. Role-Based Authorization

URL pattern	Access
`/api/users/register`, `/api/eligibility/**`	Public
`/actuator/**`, Swagger UI, `/api-docs/**`	Public
`/api/students/**`	ADMIN, STUDENT
`/api/companies/**`	ADMIN, COMPANY
`/api/job-drives/**`, `/api/applications/**`, `/api/interviews/**`, `/api/offers/**`	ADMIN, STUDENT, COMPANY
Any other request	ADMIN only

11. DTOs

DTO (Data Transfer Object) classes transfer data between the client and the application. They keep the API model separate from the database entities and are used for input validation.

DTO	Purpose
StudentRequestDTO / ResponseDTO	Student input and output.
CompanyRequestDTO / ResponseDTO	Company input and output.
JobDriveRequestDTO / ResponseDTO	Job drive input and output.
ApplicationRequestDTO / ResponseDTO	Student ID and job drive ID; application details.
InterviewRequestDTO / ResponseDTO	Interview scheduling and details.
OfferRequestDTO / ResponseDTO	Offer creation and details.
EligibilityResponseDTO	Eligibility result with message.

12. Important Business Validations

- Student and company email must be in a valid format.
- Student CGPA must be between 0 and 10.
- Minimum CGPA and salary cannot be negative.
- Required fields (name, phone, location, dates, IDs) cannot be blank.
- Applications are accepted only for OPEN job drives.
- A student must be eligible for the drive before applying.
- A student cannot apply twice for the same job drive.
- Interviews and offers must be linked to an existing application.
- Records that do not exist return a Not Found error.

13. Eligibility Check

A student is eligible for a job drive only when all four conditions are true:

- Student CGPA is greater than or equal to the drive's minimum CGPA.
- Student branch matches the required branch (ignoring case), or the drive has no branch restriction.
- The job drive status is OPEN.
- Today's date is not after the application deadline.

14. REST API Documentation

Swagger UI: http://localhost:8080/swagger-ui.html (OpenAPI JSON: http://localhost:8080/api-docs). It works only while the application is running on port 8080.

Method	Endpoint	Purpose
POST	`/api/users/register`	Register a user with a role (public)
POST	`/api/students`	Create student
GET	`/api/students`	List all students
GET	`/api/students/{studentId}`	Get student by ID

Method	Endpoint	Purpose
PUT	`/api/students/{studentId}`	Update student
DELETE	`/api/students/{studentId}`	Delete student
GET	`/api/students/branch/{branch}`	Students of a branch
GET	`/api/students/cgpa/{cgpa}`	Students with CGPA greater than or equal to value
GET	`/api/students/pagination`	Paged and sorted list (page, size, sortBy, direction)
POST	`/api/students/{studentId}/job-drives/{jobDriveId}`	Link student to a job drive
GET	`/api/students/{studentId}/job-drives`	Job drives linked to a student
POST	`/api/companies`	Create company
GET	`/api/companies`	List all companies
GET	`/api/companies/{companyId}`	Get company by ID
PUT	`/api/companies/{companyId}`	Update company
DELETE	`/api/companies/{companyId}`	Delete company
GET	`/api/companies/industry/{industry}`	Filter by industry
GET	`/api/companies/location/{location}`	Filter by location
GET	`/api/companies/pagination`	Paged and sorted list
POST	`/api/job-drives`	Create job drive
GET	`/api/job-drives`	List all job drives
GET	`/api/job-drives/{jobDriveId}`	Get job drive by ID
PUT	`/api/job-drives/{jobDriveId}`	Update job drive
DELETE	`/api/job-drives/{jobDriveId}`	Delete job drive
GET	`/api/job-drives/status/{status}`	Filter by OPEN / CLOSED
GET	`/api/job-drives/location/{location}`	Filter by location
GET	`/api/job-drives/eligible/{cgpa}`	Drives a given CGPA qualifies for
PATCH	`/api/job-drives/{jobDriveId}/close`	Close a job drive manually
GET	`/api/job-drives/pagination`	Paged and sorted list
GET	`/api/eligibility/{studentId}/{jobDriveId}`	Check student eligibility for a drive (public)
POST	`/api/applications`	Apply for a job drive
GET	`/api/applications`	List all applications
GET	`/api/applications/{applicationId}`	Get application by ID
GET	`/api/applications/student/{studentId}`	Applications of a student
GET	`/api/applications/job-drive/{jobDriveId}`	Applications for a drive
GET	`/api/applications/status/{status}`	Filter by application status

Method	Endpoint	Purpose
PUT	`/api/applications/{applicationId}/status/{status}`	Update application status
DELETE	`/api/applications/{applicationId}`	Delete application
GET	`/api/applications/pagination`	Paged and sorted list
POST	`/api/interviews`	Schedule interview
GET	`/api/interviews`	List all interviews
GET	`/api/interviews/{interviewId}`	Get interview by ID
GET	`/api/interviews/result/{result}`	Filter by PENDING / PASSED / FAILED
GET	`/api/interviews/student/{studentId}`	Interviews of a student
PUT	`/api/interviews/{interviewId}/result/{result}`	Update interview result
DELETE	`/api/interviews/{interviewId}`	Delete interview
GET	`/api/interviews/pagination`	Paged and sorted list
POST	`/api/offers`	Create offer
GET	`/api/offers`	List all offers
GET	`/api/offers/{offerId}`	Get offer by ID
GET	`/api/offers/status/{status}`	Filter by PENDING / ACCEPTED / REJECTED
GET	`/api/offers/student/{studentId}`	Offers of a student
PUT	`/api/offers/{offerId}/status/{status}`	Update offer status
DELETE	`/api/offers/{offerId}`	Delete offer
GET	`/api/offers/pagination`	Paged and sorted list

15. Complete Placement Flow

```
Register User → Create Company → Create Job Drive
        ↓
Student Registered → Check Eligibility
        ↓
Apply for Job Drive (status = APPLIED)
        ↓
Shortlisted → Interview Scheduled
        ↓
Interview Result (PASSED / FAILED)
        ↓
Application SELECTED / REJECTED
        ↓
Offer Created (PENDING) → ACCEPTED / REJECTED
```

16. Example Requests

Create Job Drive

```
POST /api/job-drives
Authorization: Basic <username:password>
{
  "jobTitle": "Java Developer",
  "description": "Backend development role",
  "minimumCgpa": 7.0,
  "requiredBranch": "CSE",
  "location": "Bengaluru",
  "driveDate": "2026-11-20",
  "applicationDeadline": "2026-11-10",
  "companyId": 1
}
```

Apply for Job Drive

```
POST /api/applications
Authorization: Basic <username:password>
{
  "studentId": 1,
  "jobDriveId": 1
}
```

Update Application Status

```
PUT /api/applications/1/status/SHORTLISTED
Authorization: Basic <username:password>
```

17. Background Jobs, Caching and Configuration

Feature	Details
Job drive scheduler	Runs every midnight and closes OPEN drives whose deadline has passed.
Placement report	Prints totals of students, companies, drives and applications every 10 seconds.
Caching	Student lookups are cached and cleared on create, update or delete.
Auditing	Created and updated date and user are filled automatically.
Pagination	Every module supports page, size, sort field and direction.
Profiles	dev (default, SQL shown), test, prod (SQL hidden, INFO logs).
Server / Database	Port 8080, MySQL `placement_db` on localhost:3306.
Swagger UI	http://localhost:8080/swagger-ui.html
Logging	Console and `PlacementApplication.log`

18. Postman Testing Order

Step	Test
1	Register an ADMIN user
2	Create a company
3	Create a job drive
4	Create a student
5	Check eligibility
6	Apply for the job drive
7	Update application status
8	Schedule an interview and update its result
9	Create an offer and update its status
10	Test filters and pagination

19. Common Errors

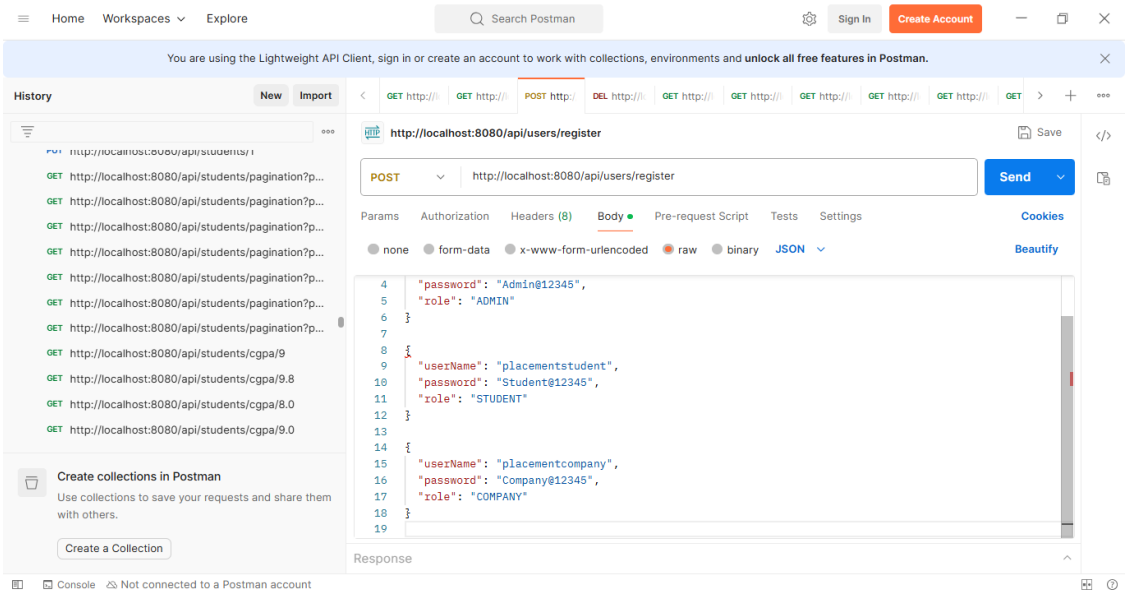
Error	Solution
401 Unauthorized	Send valid username and password using Basic Auth.
403 Forbidden	Check whether the logged-in role is allowed for that URL.
404 Not Found	The ID does not exist, or verify the URL and HTTP method.
400 Bad Request	Fix invalid input, or the business rule failed (drive closed, not eligible, already applied).
500 Internal Server Error	Check `PlacementApplication.log` for details.
Port 8080 already in use	Stop the other process or change `server.port`.
MySQL connection error	Start MySQL, create `placement_db` and verify datasource credentials.

20. Future Enhancements

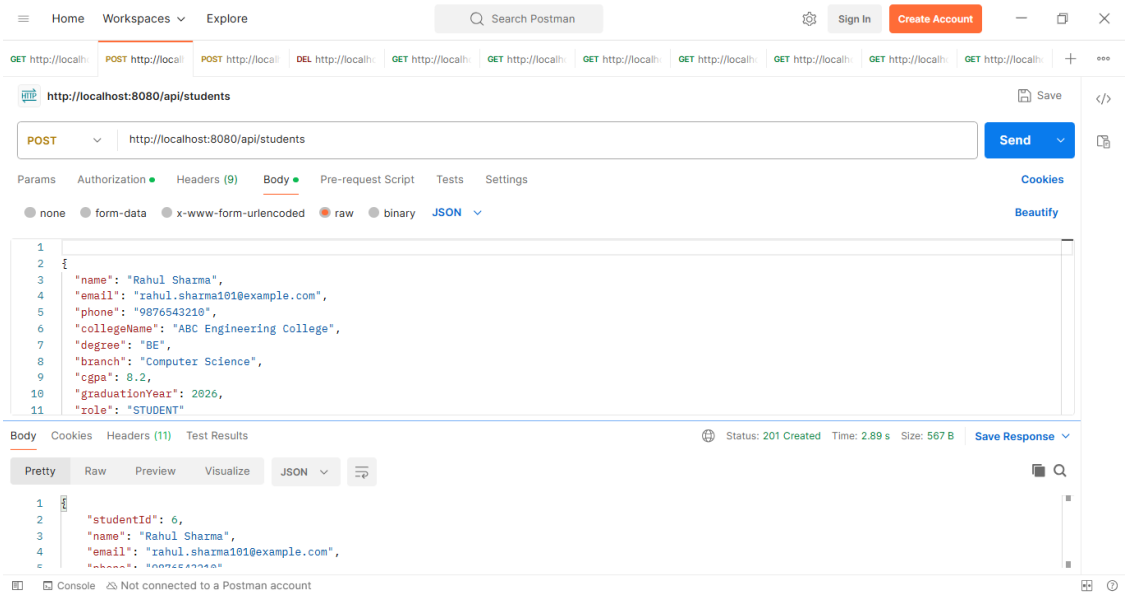
- Move database credentials to environment variables.
- Replace HTTP Basic with JWT authentication.
- Add ownership checks so students see only their own data.
- Validate application status transitions.
- Add email notifications and resume upload.

21. Snapshots of Student Placement Management System

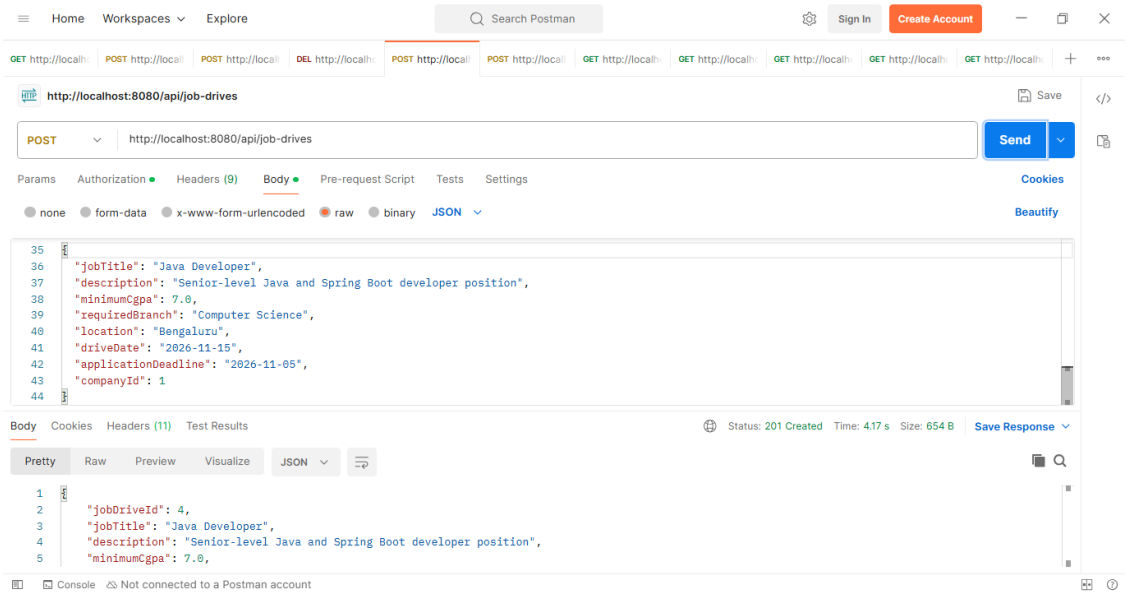
a. User Registration



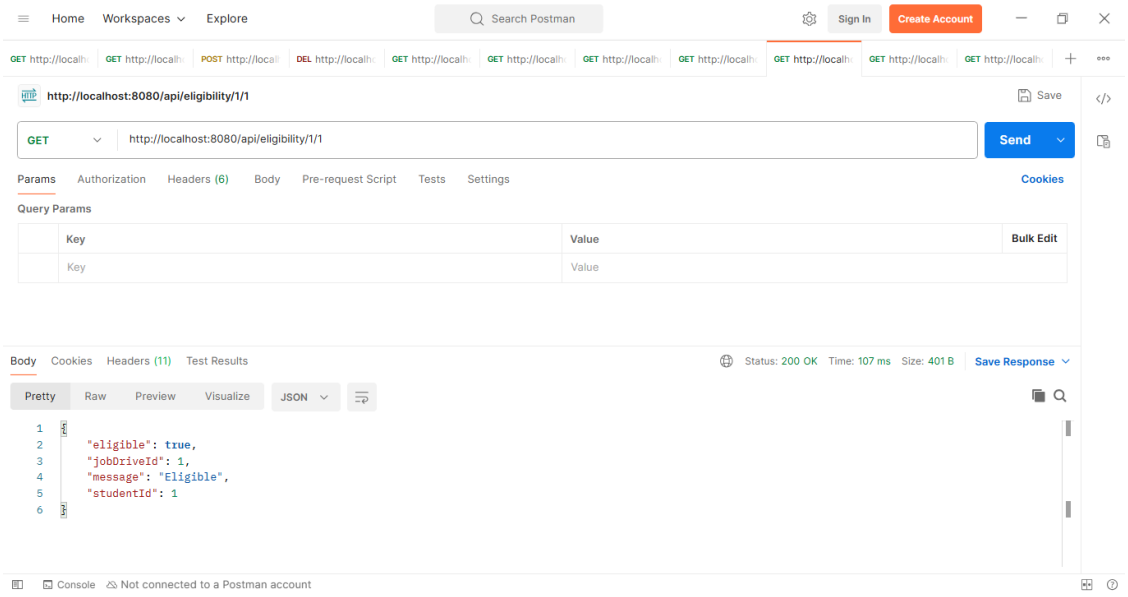
b. Student Creation



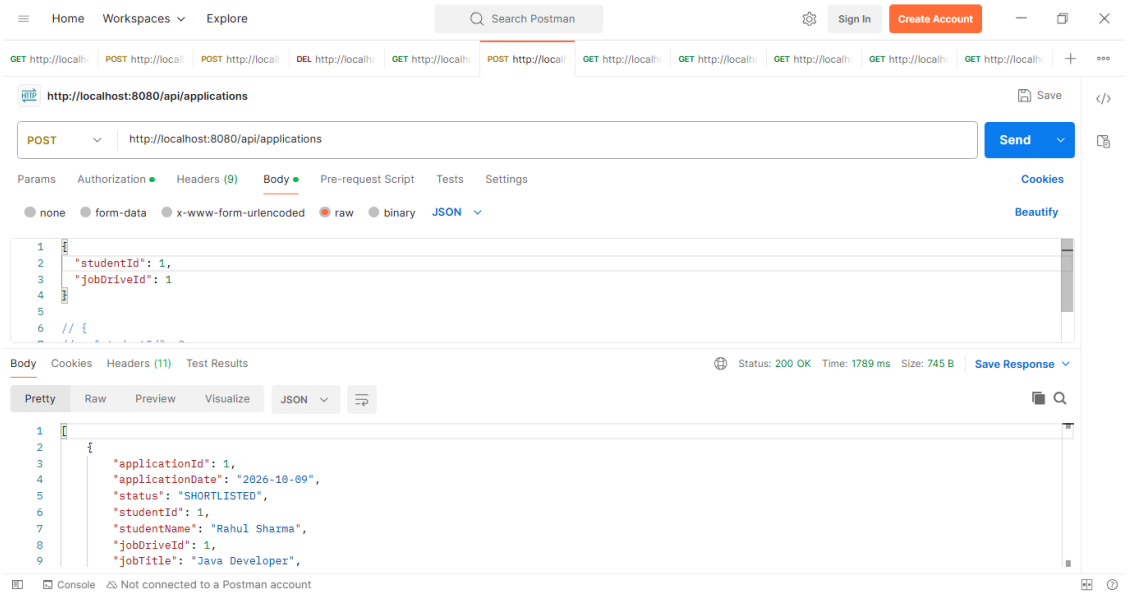
c. Job Drive Creation



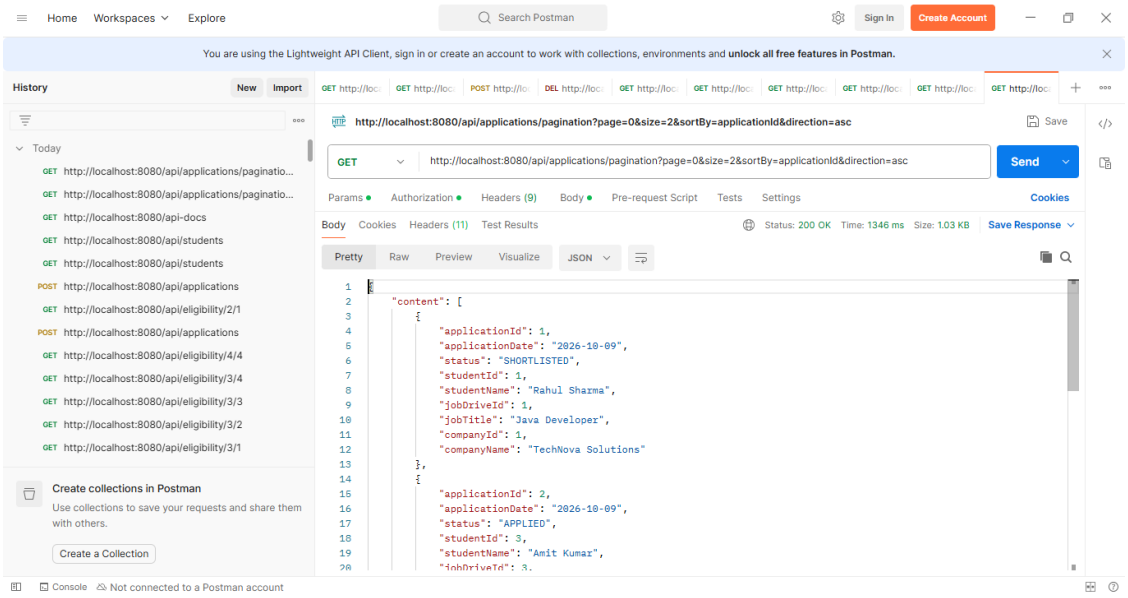
d. Eligibility Check

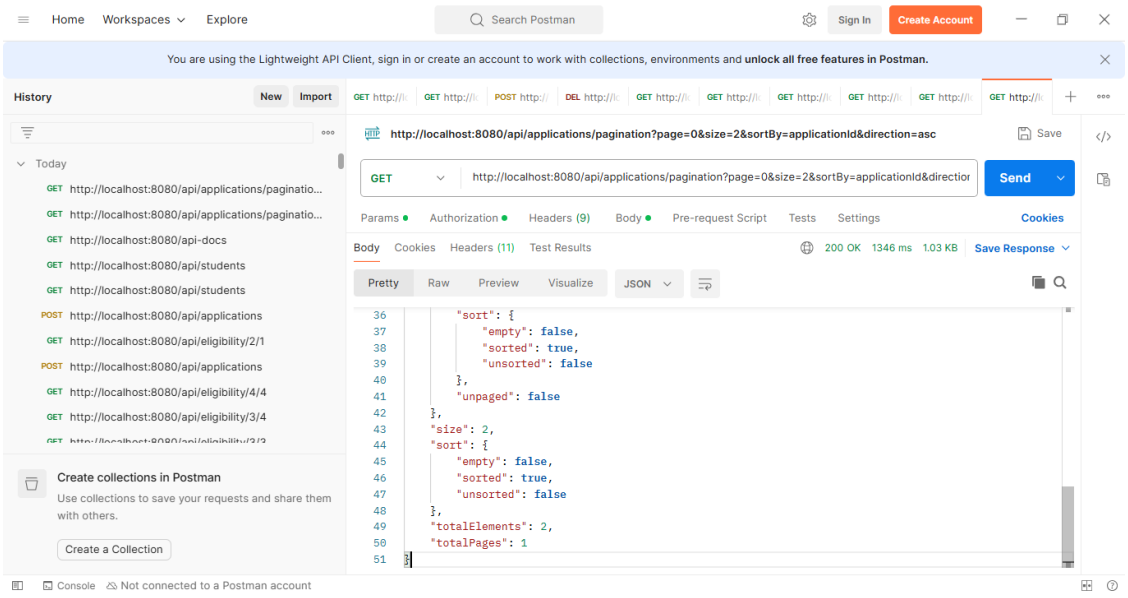


e. Apply for Job Drive

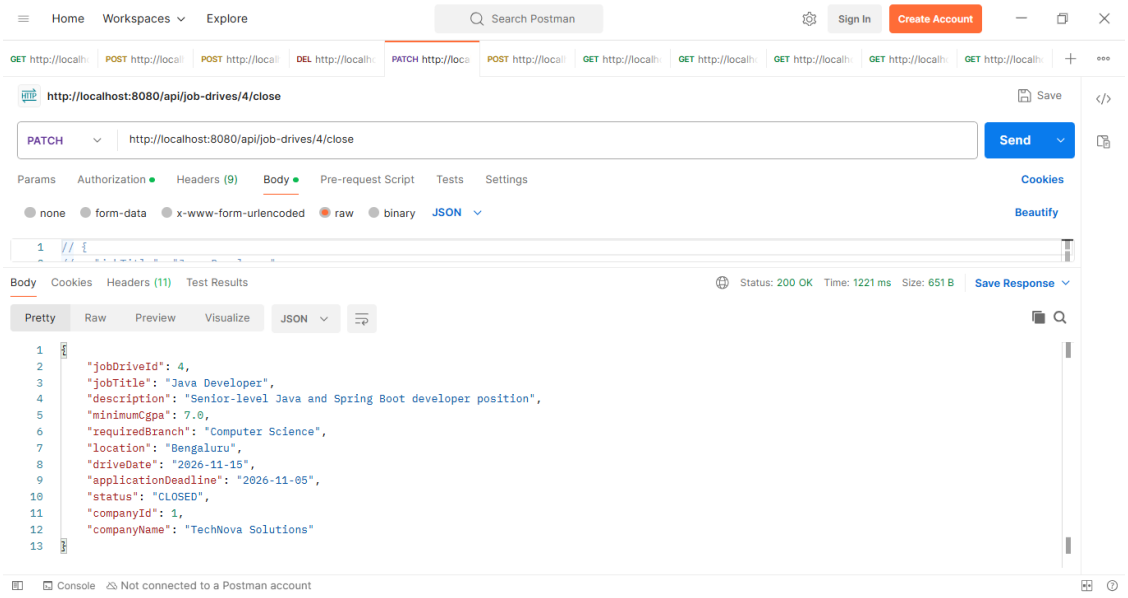


f. Pagination and Sorting of Applications

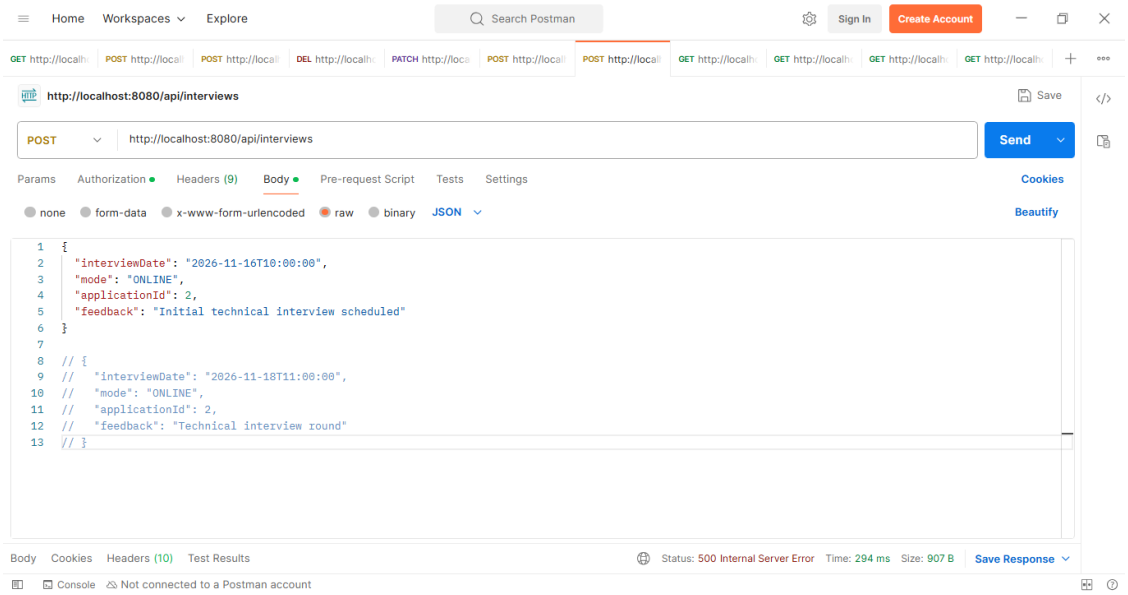




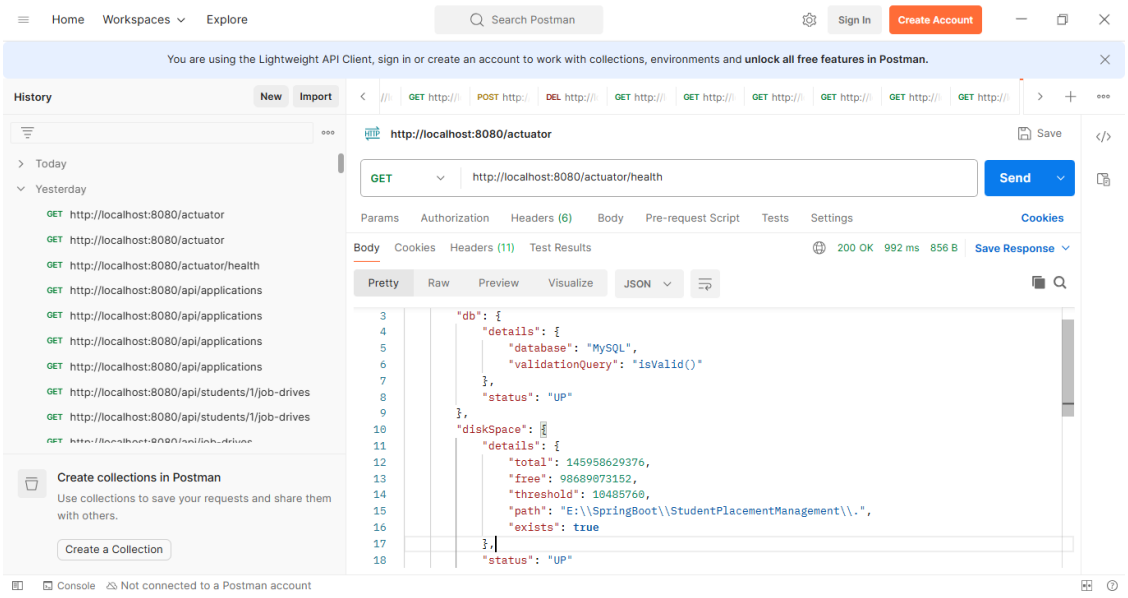
g. Close Job Drive



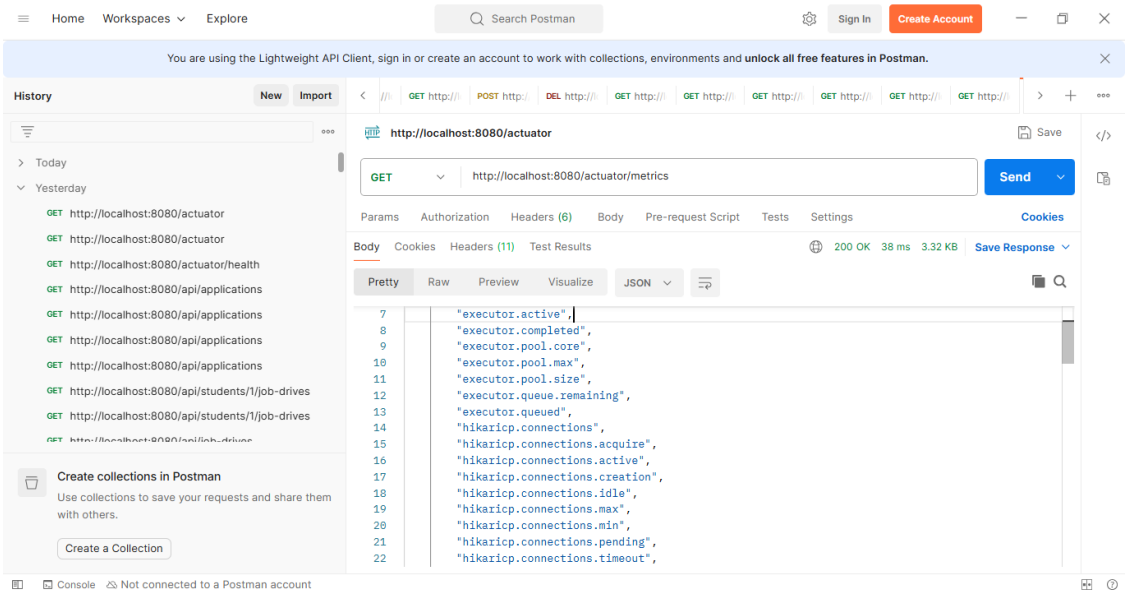
h. Schedule Interview

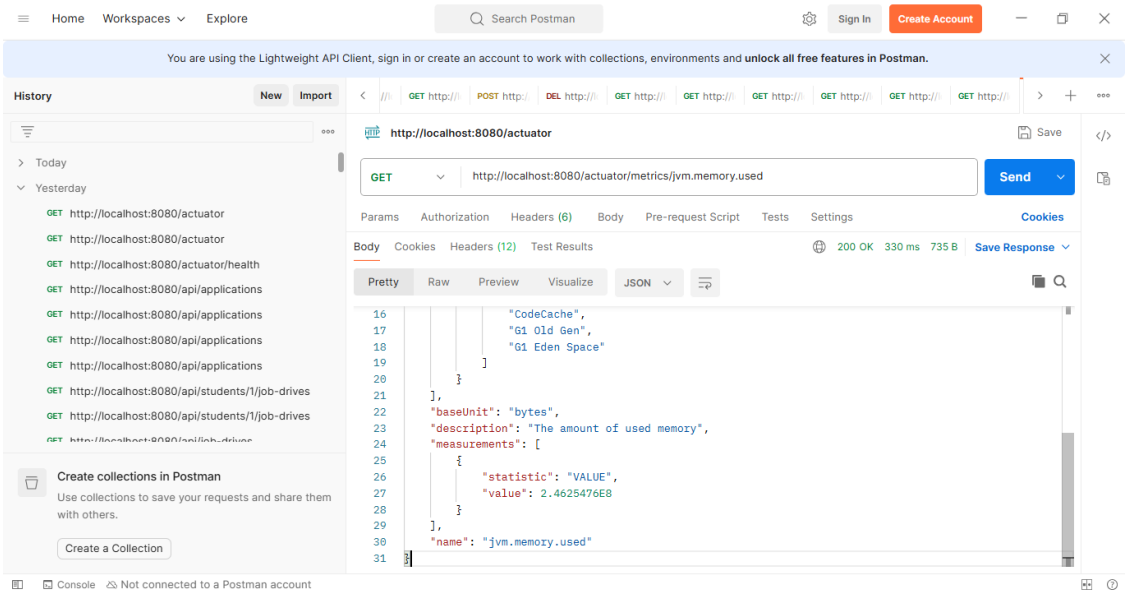


i. Application Health Check



j. Application Metrics





22. Project Summary

The Student Placement Management System demonstrates a complete backend application using Spring Boot. It combines REST APIs, JPA/Hibernate, MySQL, validation, exception handling, role-based security, automatic eligibility checking, scheduled jobs and caching. The layered design makes the project suitable for learning, demonstration and further extension.

— End of Project Documentation —